

UNIVERSIDAD AUTÓNOMA DEL PERÚ
Facultad de Ingeniería y Arquitectura | Ingeniería de Sistemas
Hoja de Trabajo - Programación Segura - Semana 1

Ciclo: VIII

Curso: Programación Segura

Tema: Introducción a la Programación Segura y Modelado de Seguridad

Estudiante: Ana Concepción Asto Arotinco

Docente: Ruben Quispe Llaccharimay

Sección A: Completar la Frase

1. La **Autenticación** es el proceso de verificar la identidad de un usuario antes de permitirle el acceso a un sistema.
2. La **Integridad** garantiza que la información no sea modificada sin autorización por parte de personas no autorizadas.
3. La **Disponibilidad** asegura que los sistemas y datos estén disponibles para los usuarios autorizados en el momento en que los necesitan.
4. Una **Vulnerabilidad** es una debilidad en el diseño, implementación o configuración de un sistema que puede ser explotada por un atacante.
5. Una **Amenaza** es cualquier circunstancia o evento con el potencial de causar daño a un activo de información.
6. El **Riesgo** se define como la combinación de la probabilidad de que ocurra un evento adverso y el impacto resultante sobre los activos.
7. OWASP son las siglas de **Open Web Application Security Project**, una fundación sin fines de lucro que publica el Top 10 de vulnerabilidades web.
8. La **Autorización** define qué recursos o acciones puede ejecutar un usuario dentro del sistema una vez que ya ha sido autenticado.
9. Un **Activo** es cualquier recurso de valor para la organización, como datos, sistemas, infraestructura o personas.
10. El principio de **Mínimo Privilegio** establece que cada usuario debe tener únicamente los permisos necesarios para realizar su trabajo.
11. La **Seguridad por Diseño** de un sistema consiste en definir los requerimientos de seguridad desde las fases iniciales del desarrollo del software.

12. Un **Control de seguridad** es un mecanismo, proceso o tecnología implementado para reducir la probabilidad o el impacto de una amenaza o vulnerabilidad.
13. La **Ingeniería social** es una técnica de ataque que manipula a las personas para obtener información confidencial aprovechando su confianza.
14. El **No repudio** garantiza que las acciones realizadas en un sistema puedan atribuirse de forma inequívoca e irrefutable a quien las ejecutó.
15. El **Modelado** de amenazas es una técnica sistemática para identificar y priorizar posibles ataques contra un sistema antes de construirlo.
16. La criptografía **Asimétrica** utiliza un par de claves—una pública y una privada— para cifrar y descifrar información de forma segura.
17. Una función de **Hash** transforma datos de cualquier tamaño en una cadena de longitud fija, siendo un proceso matemáticamente irreversible.
18. En los **Casos de uso** seguros se documenta el flujo que el sistema debe seguir para proteger una funcionalidad frente a posibles ataques o abusos.
19. La autenticación **Multifactor** (MFA) combina dos o más factores de verificación para aumentar significativamente la seguridad del acceso.
20. El ciclo de vida de desarrollo seguro (**SDLC**) integra actividades y controles de seguridad en cada fase del proceso de construcción del software.

Sección B: Relacionar Conceptos

Concepto	Definición Correspondiente
1. Amenaza	s) Evento o circunstancia con potencial de causar daño a un activo.
2. Vulnerabilidad	j) Debilidad en un sistema que puede ser aprovechada por un atacante.
3. Riesgo	m) Combinación de probabilidad de ataque e impacto sobre los activos.
4. Firewall	f) Mecanismo que filtra el tráfico de red según reglas de seguridad predefinidas.
5. IDS	h) Sistema que detecta actividad sospechosa en la red y genera alertas.

6. IPS	k) Sistema que detecta Y bloquea activamente el tráfico malicioso en la red.
7. OWASP	e) Fundación que publica el Top 10 de vulnerabilidades web más críticas.
8. Hash	n) Función que transforma datos en una cadena de longitud fija e Irreversible.
9. Cifrado	i) Proceso de transformar datos legibles en un formato ilegible mediante una clave.
10. MFA	q) Uso de dos o mas factores para verificar la identidad de un usuario.
11. Autenticación	b) Proceso de verificar que un usuario es quien dice ser.
12. Autorización	p) Mecanismo que define qué puede hacer un usuario ya autenticado.
13. Activo	d) Recurso de valor para la organización: datos, hardware, software o servicios.
14. Control de seguridad	r) Medida implementada para reducir la probabilidad o impacto de una amenaza.
15. Ataque	g) Acción deliberada que explota una vulnerabilidad para comprometer la seguridad.
16. Ingeniería social	l) Técnica que manipula a personas para revelar información confidencial.
17. Malware	a) Software malicioso diseñado para dañar, robar o comprometer sistemas.
18. SQL Injection	t) Ataque que inyecta sentencias SQL en formularios para manipular la base de datos.

19. XSS

c) Técnica que inserta código malicioso en páginas web para ejecutarse en el navegador del visitante.

20. Modelado de amenazas

o) Técnica sistemática para identificar amenazas en un sistema antes de construirlo.

Sección C: Completar el Código (Java)

Ejercicio 1: Validar longitud mínima de contraseña

```
if (password.length() < 8) { ... }
```

Ejercicio 2: Verificar si un carácter es dígito

```
if (Character.isDigit(c) == true) { ... }
```

Ejercicio 3: Verificar si un carácter es mayúscula

```
if (Character.isUpperCase(c) == true) { ... }
```

Ejercicio 4: Complejidad robusta de contraseña

```
if (password.length() >= 8 && password.matches(".*[0-9].*") &&  
password.matches(".*[A-Z].*") && password.matches(".*[@#$%^&+=].*"))
```

Ejercicio 5: Validación de Email con Expresión Regular

```
String regex = "^[\\w._%+-]+@[\\w.-]+\\.([a-zA-Z]{2,})$";  
if (email.matches(regex)) { ... }
```

Ejercicio 6: Limpieza y validación de usuario alfanumérico

```
String cleanUser = username.trim();  
if (cleanUser.matches("^[a-zA-Z0-9]{5,12}$")) { ... }
```

Ejercicio 7: Sanitización básica contra XSS

```
String safe = input.replace("<", "&lt;").replace(">",  
"&gt;").replace("\\\"", "&quot;").replace("'", "&#x27;");
```

Ejercicio 8: Control de flujo seguro por Roles

```
switch(rol) {  
    case "ADMIN": ... break;  
    default: ...  
}
```

Ejercicio 9: Control de intentos fallidos

```
if (intentos >= 3 && user.equals(lockedUser)) { return true; }
```

Ejercicio 10: Validación de dominio institucional

```
if (email.contains("@autonoma.edu.pe") && !email.equals("")) { ... }
```

Ejercicio 11: Validación robusta de campos vacíos

```
if (input.isEmpty()) { ... } else if (!input.matches("[a-zA-Z]+")) { ... }
```

Ejercicio 12: Cambio de clave forzando que no sea igual a la anterior

```
if (!nuevaClave.equals(anterior) && nuevaClave.length() >= 8) { ... }
```

Ejercicio 13: Captura de excepción en conversión de ID numérico

```
try {  
    int id = Integer.parseInt(input);  
    if (id <= 0) throw new Exception();  
} catch (NumberFormatException e) { ... }
```

Ejercicio 14: Validación de rango de edad segura

```
if (edad < 18 || edad > 100) { System.out.println("Edad fuera de rango  
permitido"); }
```

Ejercicio 15: Validación de credenciales con mensaje seguro y ambiguo

```
if (!isValidUser || !isValidPassword) { throw new  
SecurityException("Credenciales inválidas"); }
```

Sección D: Análisis de Código

Fragmento 1: Contraseña en texto plano hardcoded

Problema: Credenciales fijas en el código fuente.

Riesgo: Exposición total de accesos si el código es visible o descompilado.

Corrección: Externalizar credenciales en variables de entorno o almacén de secretos.

Fragmento 2: Validación de entrada (Whitelisting)

Problema: Estructuralmente seguro. Aplica lista blanca y remueve espacios.

Riesgo: Bajo. Mitiga inyecciones básicas.

Mejora: Agregar límite de tamaño antes de evaluar regex para evitar ataques ReDoS (Regular Expression DoS).

Fragmento 3: Inyección de parámetros sin sanitizar

Problema: Uso directo de entradas del usuario en lógica crítica.

Riesgo: Inyección de comandos, SQLi o XSS dependiendo de dónde se use.

Corrección: Aplicar validación estricta con expresiones regulares antes de procesar.

Fragmento 4: Control de acceso por rol en sesión

Problema: Seguro si el rol viene de una fuente inmutable del servidor.

Riesgo: Elevación de privilegios si el cliente puede alterar la variable 'rol'.

Corrección: Extraer y validar el rol desde un token criptográfico firmado (JWT) en el backend.

Fragmento 5: Construcción de consulta SQL por concatenación

Problema: Concatenación directa de entradas de usuario en una consulta SQL.

Riesgo: Inyección SQL (SQLi). Un atacante puede saltarse el login o borrar la BD.

Corrección: Implementar de manera obligatoria PreparedStatement con parámetros (?).

Fragmento 6: Mensaje de error demasiado específico

Problema: El error indica exactamente si falló el usuario o la clave.

Riesgo: Enumeración de usuarios activos en la plataforma.

Corrección: Usar un mensaje genérico: 'Usuario y/o contraseña incorrectos'.

Fragmento 7: Bloqueo de intentos en memoria volátil

Problema: El contador de bloqueos vive solo en el hilo de ejecución actual.

Riesgo: Fuerza bruta masiva reiniciando conexiones o hilos.

Corrección: Persistir los intentos fallidos e IPs bloqueadas en una base de datos o caché distribuida (Redis).

Fragmento 8: Consulta SQL dinámica e insegura

Problema: Inserción directa de la variable de categoría en el query SQL.

Riesgo: Inyección SQL avanzada para exfiltrar datos de otras tablas.

Corrección: Reemplazar por consultas parametrizadas configurando el valor mediante setString().

Fragmento 9: Validación robusta de políticas de claves

Problema: Seguro. Exige de forma explícita mayúsculas, números, símbolos y longitud.

Riesgo: Bajo frente a ataques automatizados de diccionario.

Mejora: Medir la entropía de la clave (ej. zxcvbn) para mitigar patrones comunes tipo 'Admin123!'.

Fragmento 10: Asignación de privilegios por parámetro del cliente

Problema: Confía ciegamente en el rol que envía el cliente o formulario.

Riesgo: Elevación de privilegios (Broken Access Control). Cualquiera puede enviarse como 'ADMIN'.

Corrección: Definir los roles en la base de datos y asignarlos internamente en el backend tras validar la identidad.

Sección E: Modelado de Amenazas - Caso Retail Plus

E.1 Tabla de Activos Críticos

ID	Nombre del Activo	Tipo / Criticidad
----	-------------------	-------------------

ACT-01	Base de Datos de Clientes y Transacciones	Información / Alta
ACT-02	Pasarela y API de Procesamiento de Pagos	Servicio / Crítica
ACT-03	Credenciales y Tokens de Acceso del Administrador	Datos de Acceso / Alta

E.2 Tabla de Amenazas

ID	Descripción de la Amenaza	Probabilidad	Impacto
AM-01	Exfiltración masiva de datos mediante Inyección SQL	Alta	Alto
AM-02	Denegación de Servicio (DDoS) contra la pasarela de ventas	Media	Alto
AM-03	Secuestro de sesiones administrativas por Fuerza Bruta	Alta	Medio

E.3 Tabla de Vulnerabilidades

ID	Vulnerabilidad Identificada (OWASP Top 10)	Mitigación Propuesta
VUL-01	A03:2021-Injection (Falta de sanitización en buscadores)	Uso Mandatorio de PreparedStatements en toda consulta.
VUL-02	A07:2021-Identification and Authentication Failures	Implementación forzosa de MFA y políticas de contraseñas.
VUL-03	A02:2021-Cryptographic Failures (Transmisión en HTTP plano)	Configuración estricta de HTTPS con TLS 1.3 y HSTS.

E.4 Tabla de Atacantes

Tipo de Atacante	Motivación Principal	Vector de Ataque Común
Cibercriminal Organizado	Económica (Venta de base de datos o Ransomware)	Inyección SQL, Phishing corporativo avanzado.
Script Kiddie	Ego, aprendizaje o sabotaje menor sin financiamiento	Uso de herramientas automatizadas de escaneo y DDoS básico.

E.5 Tabla de Controles de Seguridad

Control	Tipo de Control	Objetivo de Seguridad
Web Application Firewall (WAF)	Preventivo / Tecnológico	Filtrar tráfico de capa 7 bloqueando OWASP Top 10.
Autenticación de Doble Factor (MFA)	Preventivo / Administrativo	Evitar accesos no autorizados por robo de claves.
Monitoreo Activo de Logs y SIEM	Detectivo / Correctivo	Identificar anomalías de acceso y activar bloqueos de IPs.

Sección F: Casos de Uso Seguros

Caso de Uso 1: Inicio de Sesión Seguro

Actor(es)	Usuario autenticado / Administrador
Objetivo	Acceder al ecosistema de forma segura garantizando confidencialidad.
Precondición	El usuario posee cuenta activa y credenciales previamente registradas.
Flujo principal	1. El usuario digita sus credenciales. 2. El sistema aplica limpieza trim() y valida nulidades. 3. El sistema compara el hash contra la base de datos utilizando BCrypt. 4. El sistema genera un token firmado JWT y concede acceso.

Flujo alternativo	- Si las credenciales fallan, el sistema muestra un mensaje ambiguo genérico, aumenta el contador de fallos y bloquea al llegar a 3 intentos.
Riesgo asociado	Ataques automatizados de fuerza bruta o suplantación de identidad.
Medida de protección	Rate Limiting por IP/Usuario y exigencia de un segundo factor (MFA).

Caso de Uso 2: Registro de Nuevo Usuario

Actor(es)	Visitante / Cliente potencial
Objetivo	Crear una cuenta en el sistema de forma segura sin exponer infraestructura.
Precondición	Correo ingresado no debe existir en los registros del sistema.
Flujo principal	1. El usuario completa los datos requeridos. 2. El backend somete los datos a expresiones regulares (Whitelisting). 3. La contraseña es validada bajo directivas estrictas de entropía. 4. El sistema genera la sal (salt) y aplica hash BCrypt antes de escribir en la BD.
Flujo alternativo	Si los datos ingresados no cumplen con el formato Regex, el sistema aborta la transacción y notifica el campo específico inválido.
Riesgo asociado	: Inyección SQL en los campos de texto o creación masiva de cuentas mediante bots.
Medida de protección	Parametrización estricta de consultas y desafío CAPTCHA integrado.

Caso de Uso 3: Procesamiento de Pago

Actor(es)	Cliente autenticado
Objetivo	Realizar la transacción financiera para la compra de bienes.
Flujo principal	1. El cliente envía la solicitud de pago. 2. El sistema tokeniza la información bancaria directamente con el proveedor externo de pagos. 3. El servidor recibe únicamente el token seguro de éxito sin almacenar datos de tarjetas (CVV/PAN). 4. Se confirma la transacción y se registra en bitácora protegida.
Riesgo asociado	Intercepción de tráfico (MitM) o robo de credenciales bancarias.
Medida de protección	TLS 1.3 obligatorio, cumplimiento de estándares PCI-DSS y cabeceras HSTS activadas.

Sección G: Plan de Desarrollo Seguro

Fase SDLC	Actividades de Seguridad Requeridas (Mínimo 3 por fase)
Requerimientos	1. Identificación y mapeo de regulaciones y cumplimiento local (ej. Protección de Datos). 2. Definición de la matriz de roles, permisos y alcances del sistema. 3. Clasificación explícita de la criticidad de los activos de información.
Diseño	1. Ejecución formal del proceso de Modelado de Amenazas sobre la arquitectura. 2. Diseño de componentes bajo el principio de Mínimo Privilegio (Zero Trust). 3. Especificación técnica de algoritmos criptográficos y gestión de llaves.
Desarrollo	1. Configuración y ejecución automatizada de herramientas SAST (Análisis Estático). 2. Uso estricto de librerías y frameworks actualizados libres de CVEs conocidos. 3. Auditorías cruzadas de código (Peer Review) enfocadas en OWASP Top 10.
Pruebas	1. Implementación de pruebas automatizadas DAST (Análisis Dinámico) en entornos espejo. 2. Pruebas de penetración dirigidas (Pentesting de Caja Negra y Gris). 3. Pruebas de resistencia de formularios mediante inyección de fallas (Fuzzing).
Despliegue	1. Procesos de Hardening (bastionado) sobre los servidores y bases de datos. 2. Almacenamiento seguro de secretos productivos mediante bóvedas digitales (Key Vault). 3. Aseguramiento de canalizaciones CI/CD firmando artefactos en el pipeline DevSecOps.
Monitoreo	1. Centralización de registros operativos (logs) en una herramienta SIEM dedicada. 2. Configuración de alertas automáticas ante comportamientos anómalos o picos de fallos.

3. Escaneos e implementación periódica de parches de seguridad en producción.

Sección H: Mini Reto de PC - Login Seguro en Java

H.1 Código Fuente Implementado

```
import java.util.HashMap;
import java.util.Map;

public class LoginService {

    private static final int MAX_ATTEMPTS    = 3;
    private static final long LOCK_DURATION = 15 * 60 * 1000; // 15
    minutos

    private Map<String, User> userStore      = new HashMap<>();
    private SessionManager   sessionManager = new SessionManager();
    private AuditLog          auditLog       = new AuditLog();

    // Registra un nuevo usuario
    public void registerUser(String username, String password) {
        if (username == null || username.isBlank())
            throw new IllegalArgumentException("Usuario inválido");
        if (!isPasswordStrong(password))
            throw new IllegalArgumentException("Contraseña débil: mínimo 8
chars, 1 mayúscula, 1 número");

        String sanitized = sanitize(username);
        userStore.put(sanitized, new User(sanitized, password));
        auditLog.log("REGISTRO", sanitized, "OK");
    }

    // Intento de login; devuelve token de sesión o null
    public String login(String username, String password, String
ipAddress) {
        String sanitized = sanitize(username);
        User user = userStore.get(sanitized);

        if (user == null) {
            auditLog.log("LOGIN", sanitized, "FALLO - usuario no existe
[IP: " + ipAddress + "]);
            return null;
        }

        // Verifica si la cuenta está bloqueada
        if (user.isLocked()) {
            long elapsed = System.currentTimeMillis() -
user.getLockTime();
            if (elapsed < LOCK_DURATION) {
                long remaining = (LOCK_DURATION - elapsed) / 1000;
                auditLog.log("LOGIN", sanitized, "BLOQUEADO [" +
remaining + "s restantes]");
            }
        }
    }
}
```

```

        System.out.println("Cuenta bloqueada. Espera " +
remaining + " segundos.");
        return null;
    }
    user.unlock(); // Desbloqueo automático tras expirar el
tiempo
}

// Verifica contraseña
if (!PasswordUtil.verifyPassword(password,
user.getPasswordHash(), user.getSalt())) {
    user.incrementFailedAttempts();
    auditLog.log("LOGIN", sanitized,
        "FALLO - intento " + user.getFailedAttempts() + "/" +
MAX_ATTEMPTS);

    if (user.getFailedAttempts() >= MAX_ATTEMPTS) {
        user.lock();
        auditLog.log("LOGIN", sanitized, "CUENTA BLOQUEADA por
exceso de intentos");
        System.out.println("Cuenta bloqueada por " +
(LOCK_DURATION / 60000) + " minutos.");
    }
    return null;
}

// Login exitoso
user.resetFailedAttempts();
String token = sessionManager.createSession(sanitized);
auditLog.log("LOGIN", sanitized, "ÉXITO - token generado [IP: " +
ipAddress + "]");
return token;
}

// Logout seguro
public void logout(String token) {
    String user = sessionManager.getUserFromToken(token);
    sessionManager.invalidateSession(token);
    if (user != null) auditLog.log("LOGOUT", user, "sesión cerrada");
}

// Comprueba una sesión activa
public boolean validateSession(String token) {
    return sessionManager.isValidSession(token);
}

// --- Utilidades privadas ---

private String sanitize(String input) {
    // Elimina caracteres no alfanuméricos para prevenir inyecciones
    return input == null ? "" : input.replaceAll("[^a-zA-Z0-9_\\-]",
"").toLowerCase();
}

private boolean isPasswordStrong(String password) {
    if (password == null || password.length() < 8) return false;

```

```

        boolean hasUpper =
password.chars().anyMatch(Character::isUpperCase);
        boolean hasDigit =
password.chars().anyMatch(Character::isDigit);
        return hasUpper && hasDigit;
    }
}

```

H.2 — Captura de ejecución (muestra: éxito · contraseña corta · sin número · sin mayúscula)

H.3 Análisis y Justificación de Líneas Clave

Nº	Fragmento de Código	¿Qué hace y por qué es importante para la seguridad?
1	<pre>boolean tieneNumero = password.matches(".*[0-9].*");</pre>	Evalúa mediante expresiones regulares la presencia indispensable de dígitos numéricos en la cadena. Eleva la complejidad matemática de la clave, disminuyendo drásticamente la efectividad de ataques de diccionario o ataques basados en palabras comunes.
2	<pre>if (intentosFallidos >= MAX_INTENTOS) { ... }</pre>	Implementa un control de tasa o límite de intentos (Rate Limiting). Rompe el ciclo e inhabilita la continuidad de la ejecución protegiendo el endpoint

de autenticación contra ataques automatizados de fuerza bruta.

3 `boolean longitudValida =
password.length() >= 8;`

Establece una barrera de longitud física mínima de 8 caracteres. Incrementa de manera directa el espacio de búsqueda y la entropía de la clave, impidiendo la configuración de contraseñas triviales fácilmente vulnerables.